

Cairo University

Faculty of Computers and Artificial Intelligence



CS251

Introduction to Software Engineering

Masroofy

Software Design Specification (SDS)

Prepared by:

The Budgeteers Team

Instructor:

Dr. Mohamed El-Ramly

Version:

Final Version

Date:

May 2026



CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

Team Members

ID	NAME	EMAIL	MOBILE
20242375	Nawal Mahmoud Abdel Alem	20242375@stud.fci-cu.edu.eg	0114737776
20240253	Soussana Ishak Habib	20240253@stud.fci-cu.edu.eg	0121143345
20240001	Abanob Ihab Marcos	20240001@stud.fci-cu.edu.eg	0127760688
20250905	Youssef Osama Sami Azmi	20250905@stud.fci-cu.edu.eg	0103225692

MAY 2026



CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

Contents

Team	2
Document Purpose and Audience	4
System Models	
I. Architecture Diagram	4
II. Class Diagram(s)	5
III. Class Descriptions	6
IV. Sequence Diagrams	7
Class - Sequence Usage Table	14
V. State Diagram	14
VI. SOLID Principles	14
VII. Design Patterns	15
Tools	16
Ownership Report	16



CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

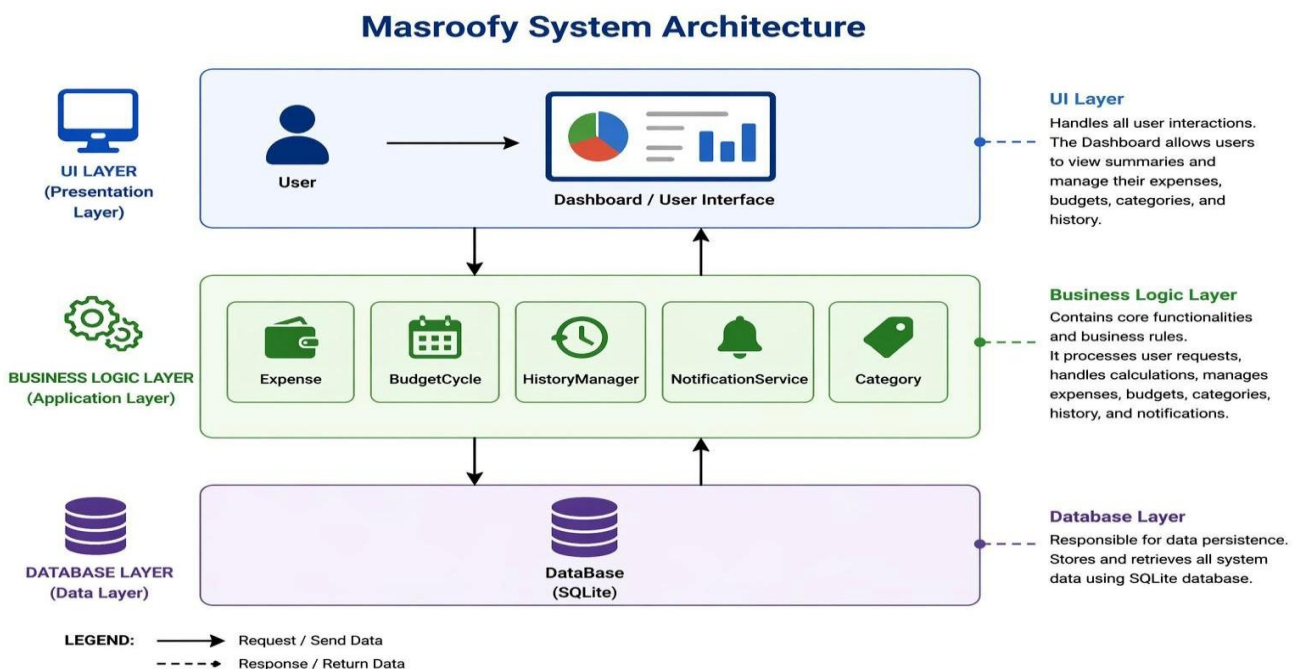
Document Purpose and Audience

Purpose: This document provides the technical design specifications for the Masroofy application. It describes the system architecture and class structures to guide the developers during the implementation phase.

Audience: The primary audience includes the Course Instructor (Dr. M. El-Ramly), the TA, and the development team and Software Testers.

System Models

I. Architecture Diagram

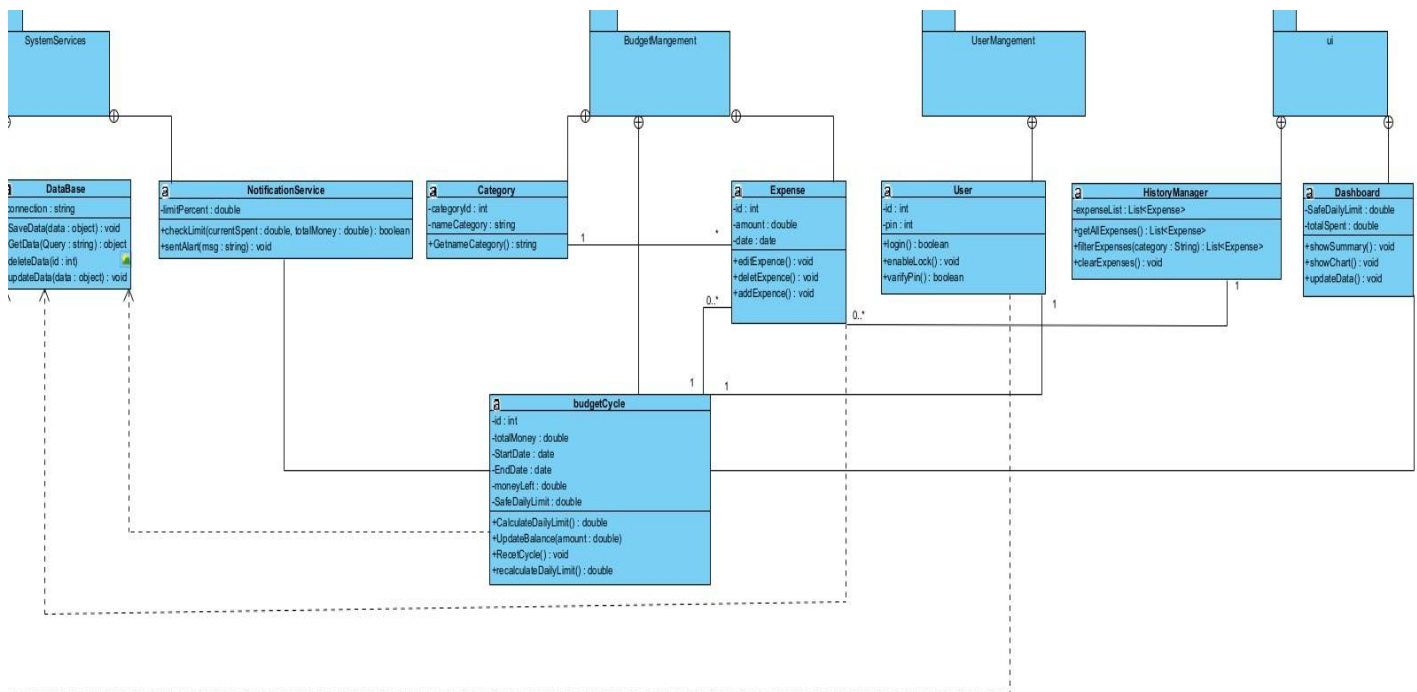




CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

II. Class Diagram(s)





CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

III. Class Descriptions

Class ID	Class Name	Description & Responsibility
1.C1	DataBase	Responsible for storing and managing all system data. Provides operations for saving, retrieving, updating, and deleting data efficiently..
2.C2	NotificationService	Monitors budget usage and sends notifications when spending limits are exceeded. Ensures the user is alerted about important financial events..
3.C3	Category	Represents an expense category such as food, transport, or bills. Helps organize expenses and supports category-based filtering.
4.C4	Expense	Represents a financial transaction including amount, date, and category. Provides functionality to add, edit, and delete expense records.
5.C5	BudgetCycle	Represents a budgeting period (e.g., monthly). Calculates spending limits and tracks the user's budget within a specific timeframe.
6.C6	User	Represents the system user and manages authentication and personal data. Allows interaction with expenses, budgets, and system features..
7.C7	HistoryManager	Manages the history of user expenses. Allows retrieving, filtering, and clearing expense records.
8.C8	Dashboard	Displays summarized financial data such as total spending and budget status. Provides an overview to help users make better financial decisions.

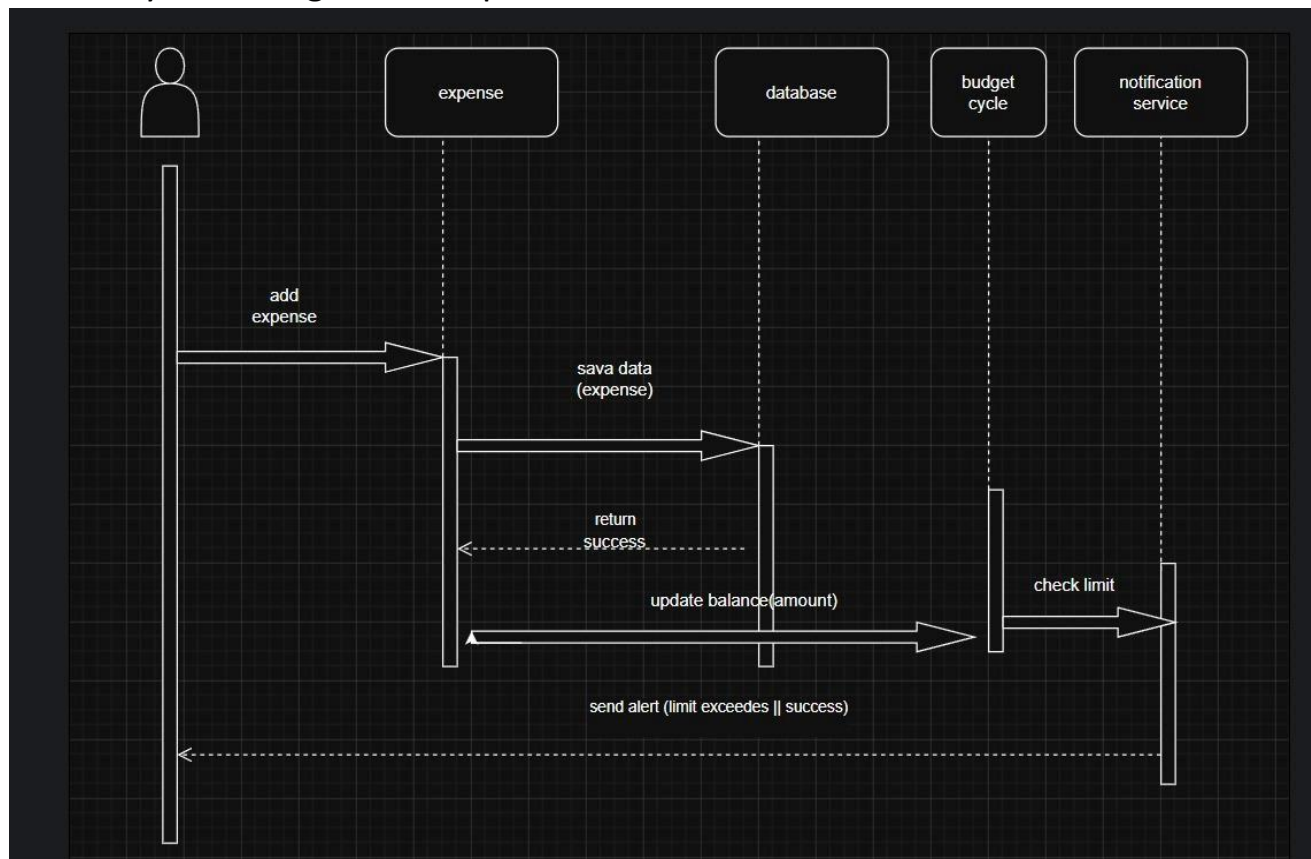


CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

IV. Sequence diagrams

User Story 1: Adding a New Expense

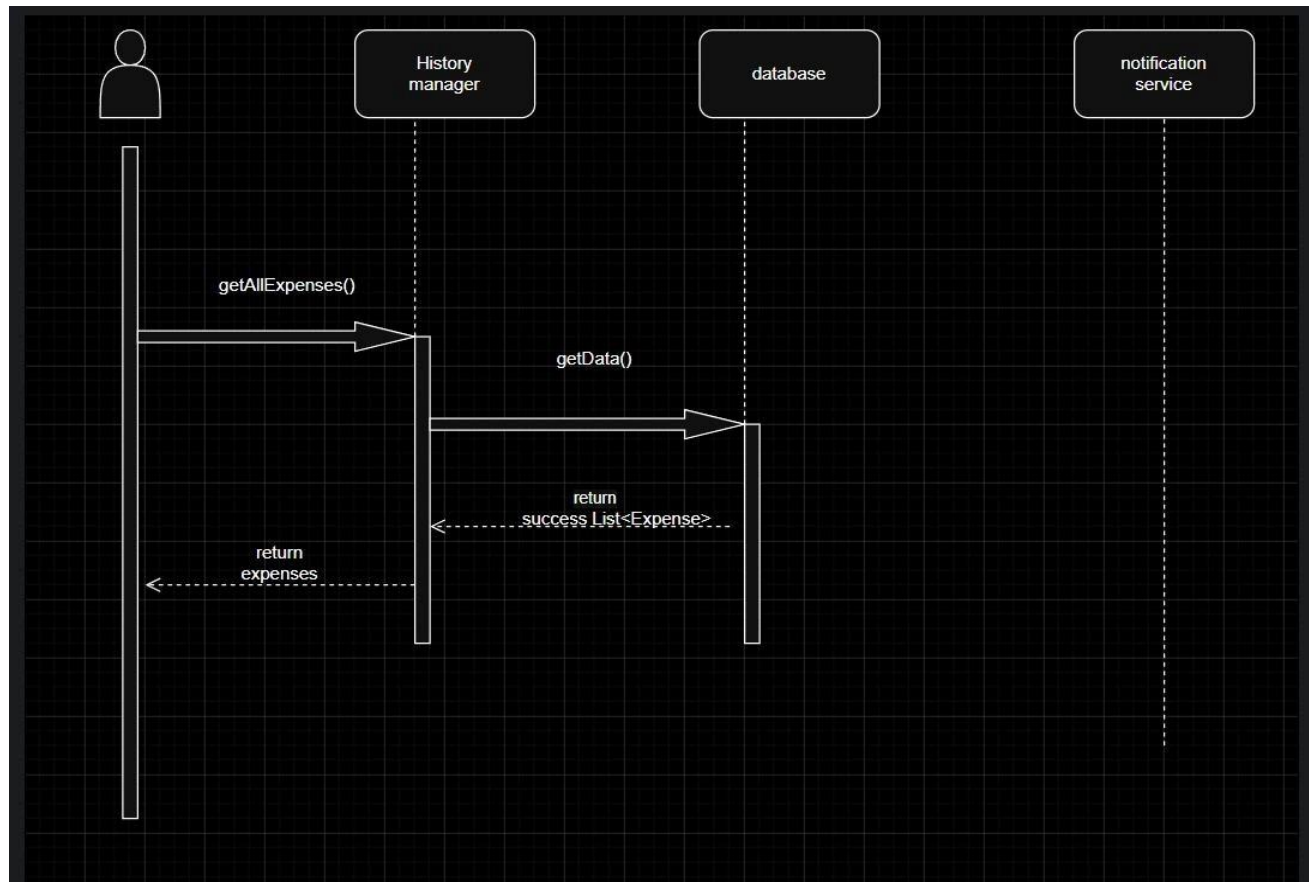




CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

User Story 2: Viewing Expense History

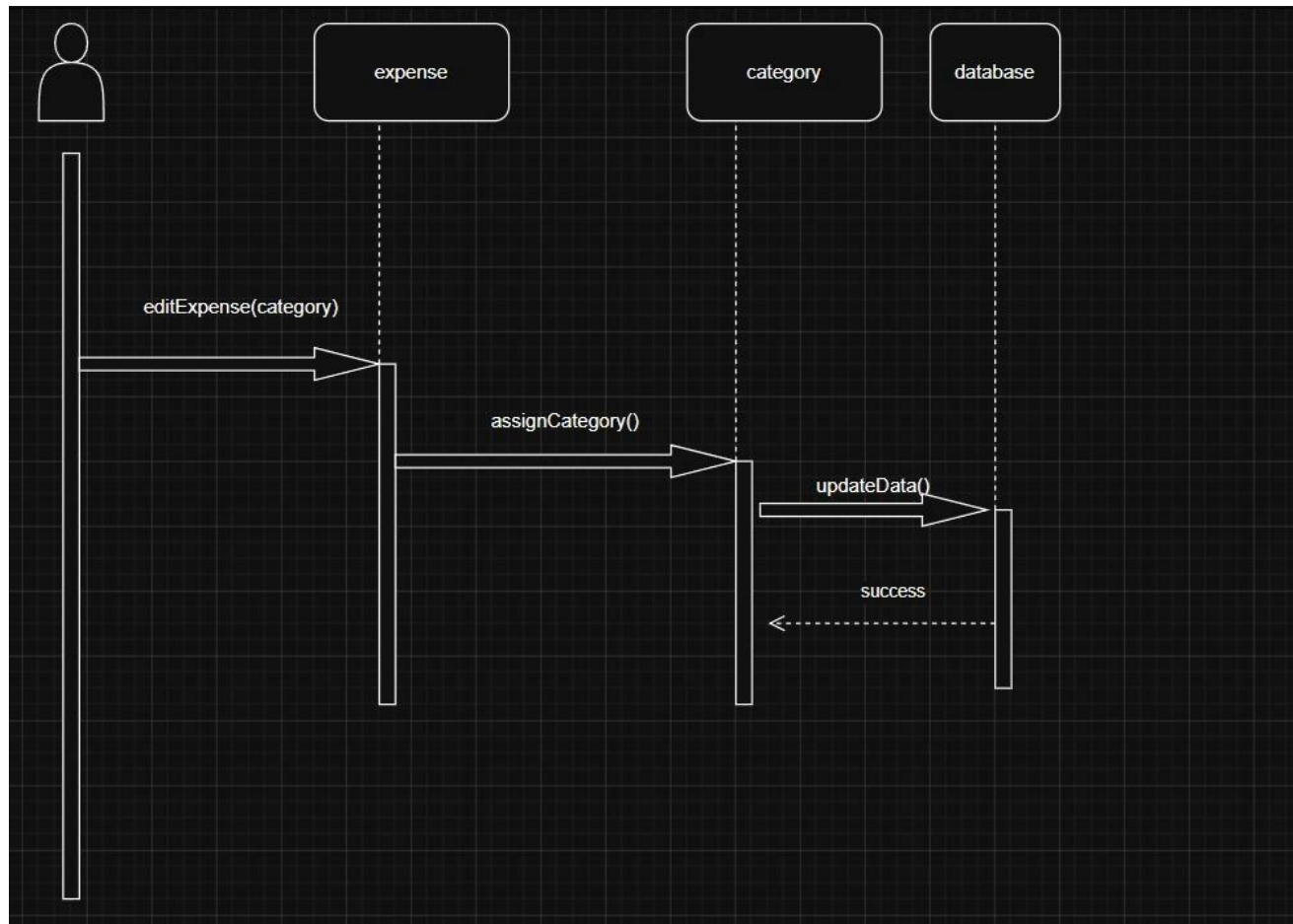




CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

User Story 3: Editing Expense Category

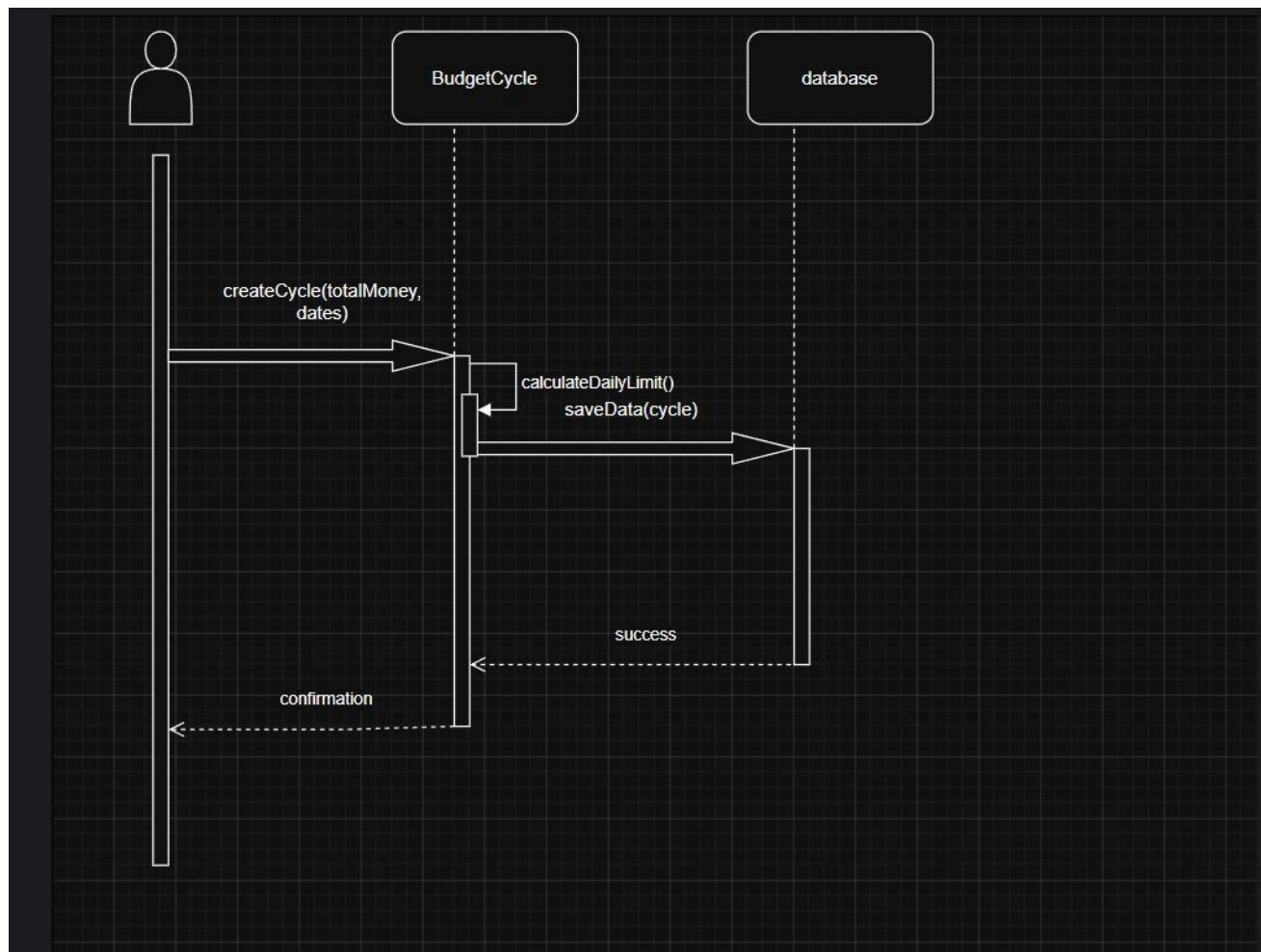




CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

User Story 4: Creating a Budget Cycle

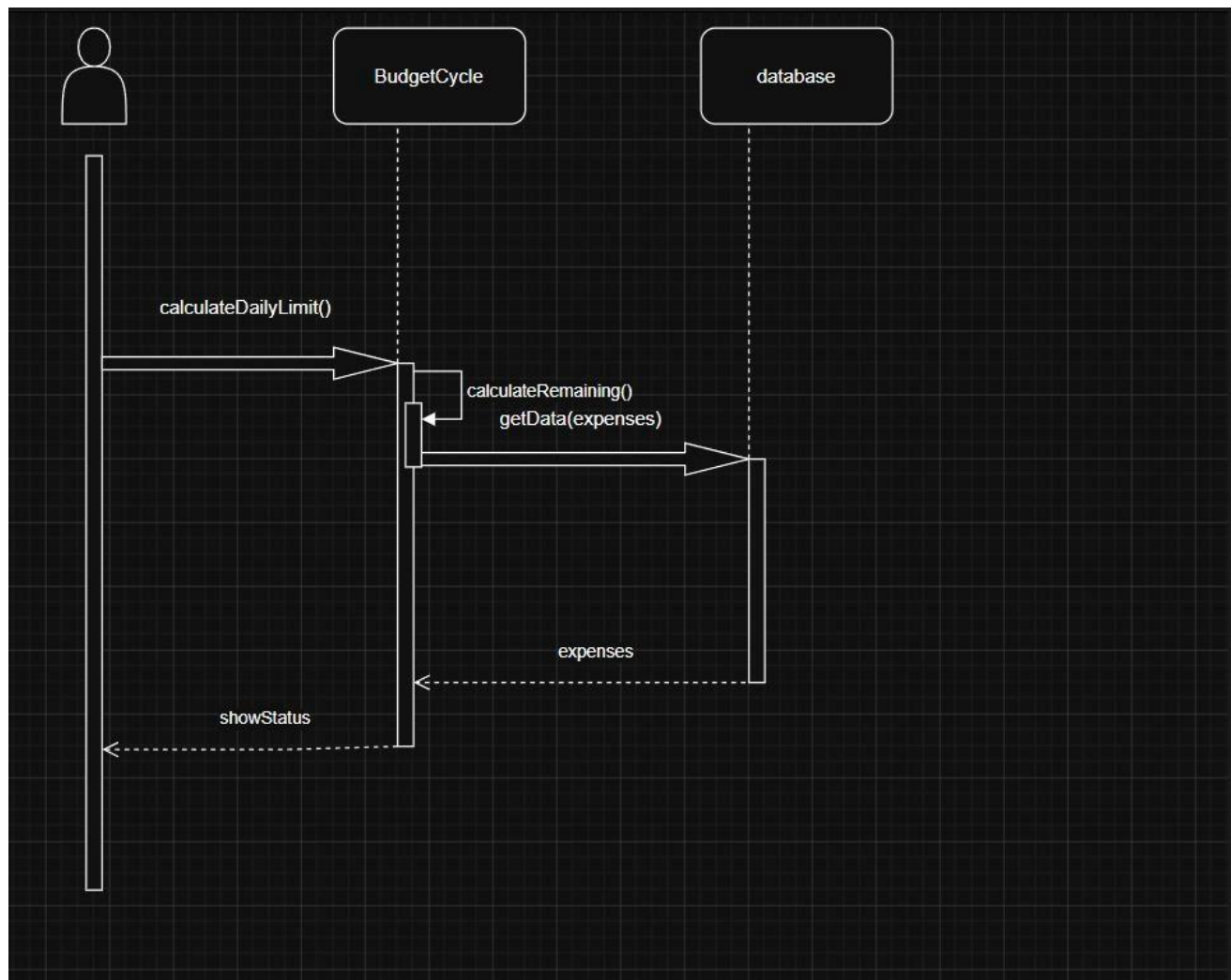




CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

User Story 5: Calculating Daily Limit And Status

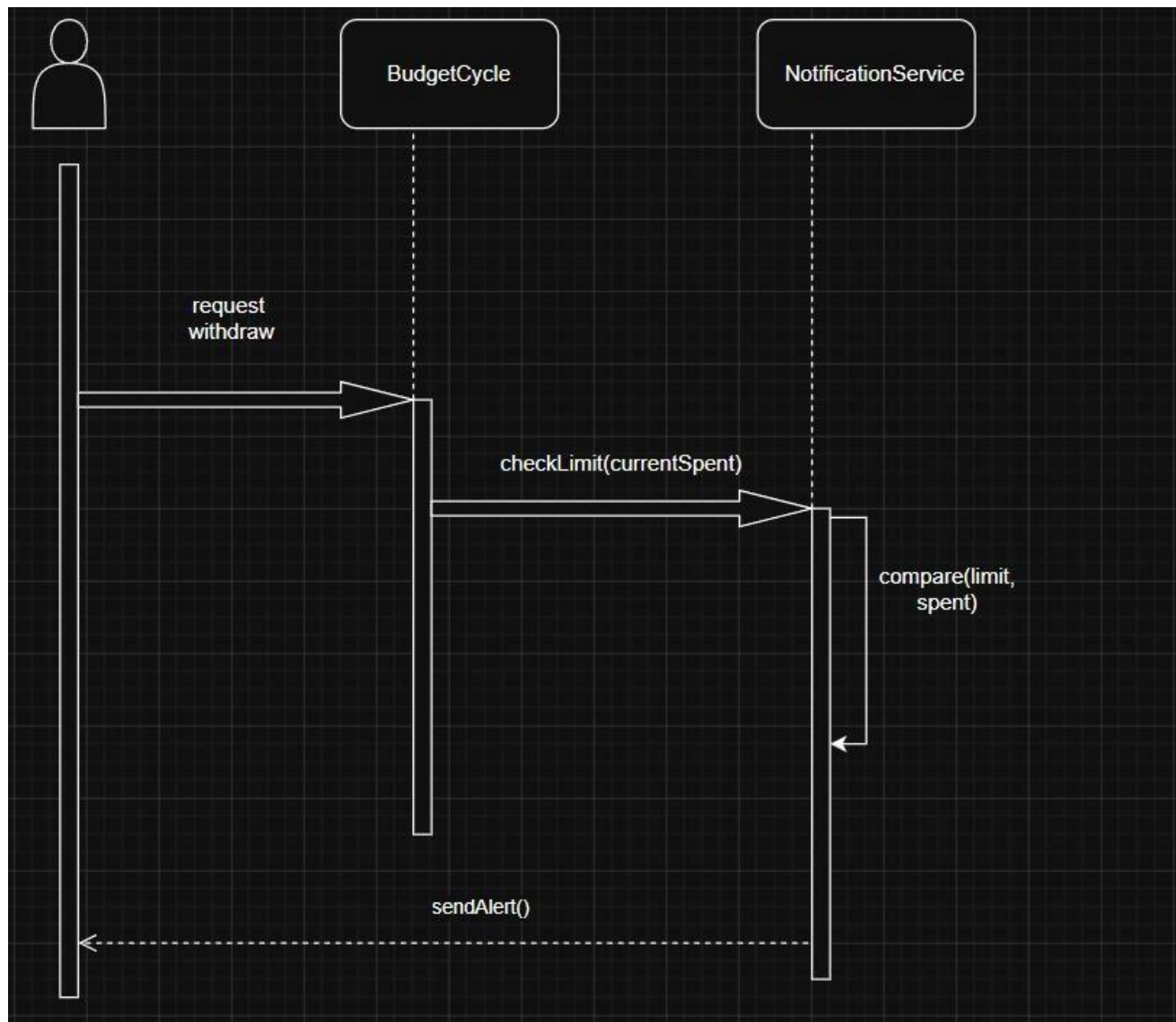




CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

User Story 6: Budget Limit Notifications

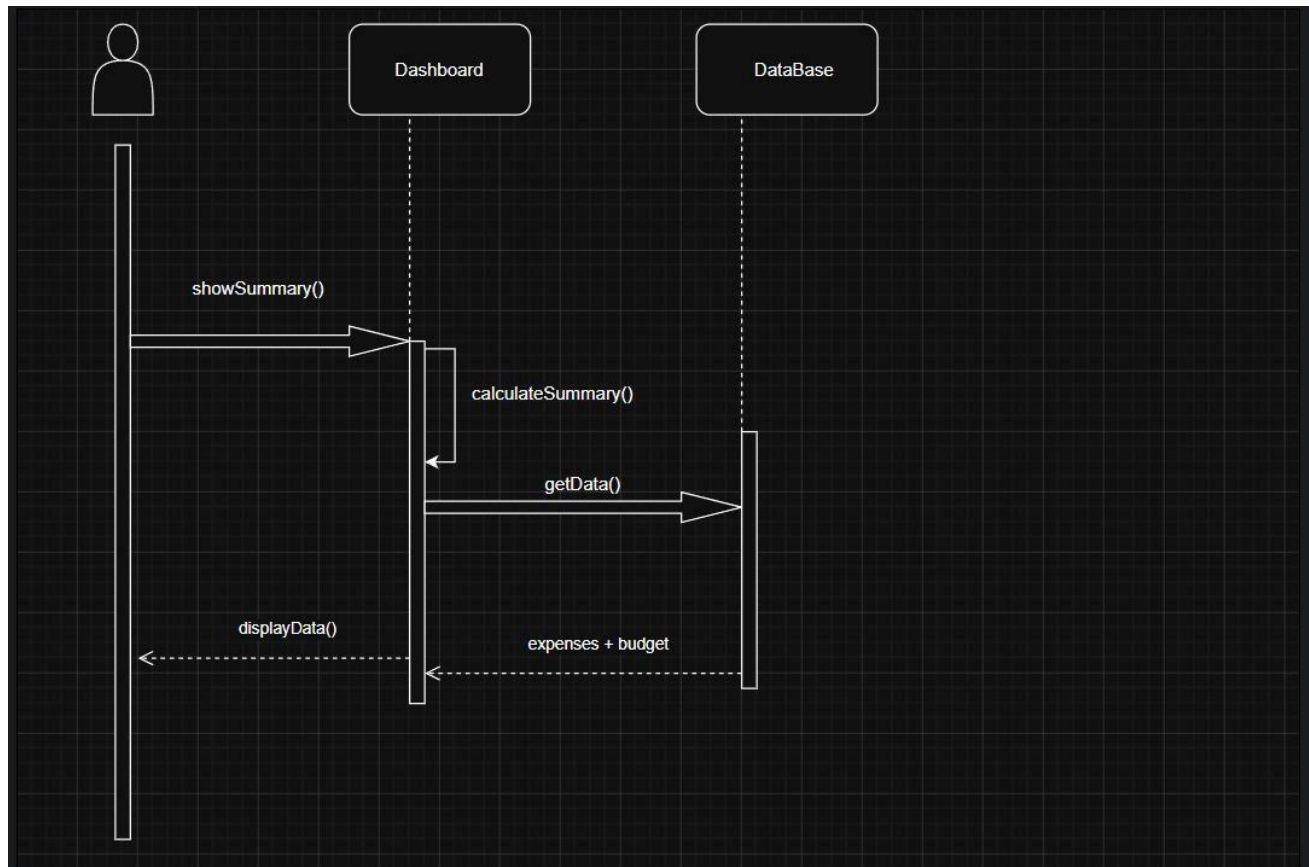




CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

User Story 7: Displaying Financial Summary





CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

Class - Sequence Usage Table

Sequence Diagram	Classes Used	All Methods Used
Add Expense	User, Expense, BudgetCycle	addExpense(), updateBudget()
View History	User, HistoryManager	getHistory()
Edit Category	User, Category	updateCategory()
Create Budget	User, BudgetCycle	createBudget()
Check Limit	BudgetCycle, NotificationService	calculateLimit(), checkStatus()
Send Notification	NotificationService, User	sendNotification()
View Dashboard	User, Dashboard	getSummary()

V. State Diagram



This state diagram represents the lifecycle of a budget cycle in the Masroofy system. It shows how the system transitions between different states such as Created, Active, Warning, Limit Exceeded, and Closed based on user spending behavior and time progression.



CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

VI. SOLID Principles

The system design follows SOLID principles to ensure maintainability, scalability, and flexibility.

1. Single Responsibility Principle (SRP):

Each class in the system has a single responsibility. For example, the Expense class handles expense data, while NotificationService is responsible for sending alerts.

2. Open/Closed Principle (OCP):

The system is designed to be extendable without modifying existing code. New features can be added by extending classes rather than changing them.

3. Liskov Substitution Principle (LSP):

Subclasses can replace their parent classes without affecting system behavior, ensuring consistency across the system.

4. Interface Segregation Principle (ISP):

The system avoids large interfaces by providing smaller, more specific interfaces tailored to client needs.

5. Dependency Inversion Principle (DIP):

High-level modules depend on abstractions rather than concrete implementations, improving flexibility and testability.



CS251: <The Budgeteers> Project: <Masroofy>

Software Design Specification

VII. Design Patterns

The system uses some design patterns to make the design more organized and easier to manage.

1. Singleton Pattern:

This pattern is used in the Database class to make sure that only one database object exists in the system.

2. Observer Pattern:

This pattern is used in the NotificationService. It helps the system send notifications to the user when the budget changes or exceeds the limit.

3. Factory Pattern:

This pattern is used to create Expense objects in a flexible and simple way instead of creating them directly.

Tools

Ownership Report

Item	Owners
Nawal Mahmoud Abdel Alem	Class Responsibility Table and organize the SDS
Soussana Ishak Habib	Class Diagram
Youssef Osama Sami Azmi	Architecture diagram +3 Sequence diagram
Abanob Ihab Marcos	4 Sequence diagram